

K8s-10

1. Setup eks cluster using eksctl

- Install AWS CLI

```
root@Master:~# sudo apt update
sudo apt install awscli -y
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease [1
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease
Fetched 933 kB in 3s (219 kB/s)
62 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
awscli

Installing dependencies:
docutils-common libimagequant0 liblcms2-2 libpaper2 libwebp7 python3-colorama python3-prompt-toolkit python3-wcwidth
libdeflate0 libjbig0 liblerc4 libraqm0 libwebpdemux2 python3-docutils python3-roman-numerals python3-ruamel.yaml
libgraphite2-3 libjpeg-turbo8 libopenjp2-7 libsharpyuv0 libwebpmux3 python3-olefile python3-ruamel.yaml
libharfbuzz0b libjpeg9 libpaper-utils libtiff6 python3-awscli python3-pil python3-ruamel.yaml.clib

Suggested packages:
liblcms2-utils docutils-doc fonts-linuxlibertine | ttf-linux-libertine texlive-lang-french texlive-latex-base texlive-latex-recommended

Summary:
Upgrading: 0, Installing: 30, Removing: 0, Not Upgrading: 62
Download size: 16.0 MB
Space needed: 148 MB / 12.5 GB available
```

- Configure AWS Credentials

```
root@Master:~# aws configure
AWS Access Key ID [None]: AKIAVMCBDU3Z3LGKTLPV
AWS Secret Access Key [None]: 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
Default region name [None]: us-east-1
Default output format [None]: json
root@Master:~# aws sts get-caller-identity
```

- Install eksctl

```
aws CLI/2.51.93 Python/3.11.4 Linux/7.0.0 2000 AWS Source/200 64-ubuntu20
root@Master:~# curl --silent --location "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_$(uname
root@Master:~# sudo mv eksctl /usr/local/bin
root@Master:~# eksctl version
0.229.0
root@Master:~#
root@Master:~# aws configure
```

- Create EKS Cluster and Update kubeconfig

```
root@Master:~# eksctl create cluster \
--name demo-eks-cluster \
--region us-east-1 \
--nodegroup-name linux-nodes \
--node-type t3.medium \
--nodes 2 \
--nodes-min 2 \
--nodes-max 3 \
--managed
```

```
root@Master:~# aws eks update-kubeconfig \
--region us-east-1 \
--name demo-eks-cluster
```

- Verify current context

```
root@Master:~# kubectl config current-context
arn:aws:eks:us-east-1:369503741683:cluster/demo-eks-cluster
root@Master:~# kubectl get nodes
No resources found
```

- Create Managed Node Group

```
root@Master:~# eksctl create nodegroup \
  --cluster demo-eks-cluster \
  --region us-east-1 \
  --name linux-nodes \
  --node-type t3.micro \
  --nodes 1 \
  --nodes-min 1 \
  --nodes-max 1 \
  --managed
```

- Verify EC2 Instance

```
root@Master:~# aws ec2 describe-instances \
  --region us-east-1 \
  --filters "Name=tag:eks:nodegroup-name,Values=linux-nodes" \
  --query "Reservations[].Instances[0].{ID:InstanceId,State:State.Name,PrivateIP:PrivateIpAddress}" \
  --output table
```

ID	PrivateIP	State
i-0ea3012d832d34f96	192.168.19.10	running

- Check Node Status

```
root@Master:~# kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-19-10.ec2.internal	NotReady	<none>	2m9s	v1.34.9-eks-8f14419

```
root@Master:~# kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-19-10.ec2.internal	NotReady	<none>	3m13s	v1.34.9-eks-8f14419

- Verify eks

```
root@Master:~# aws eks list-addons \
  --cluster-name demo-eks-cluster \
  --region us-east-1
```

```
{
  "addons": []
}
```

- Install Core Add-ons - Amazon VPC CNI, kube-proxy, Core DNS

```
root@Master:~# aws eks create-addon \
  --cluster-name demo-eks-cluster \
  --addon-name vpc-cni \
  --region us-east-1
```

```
root@Master:~# aws eks create-addon \
  --cluster-name demo-eks-cluster \
  --addon-name kube-proxy \
  --region us-east-1
```

```
root@Master:~# aws eks create-addon \
  --cluster-name demo-eks-cluster \
  --addon-name coredns \
  --region us-east-1
```

- Now verify

```
root@Master:~# aws eks list-addons \
  --cluster-name demo-eks-cluster \
  --region us-east-1
{
  "addons": [
    "coredns",
    "kube-proxy",
    "vpc-cni"
  ]
}
```

- Verify System Pods

```
root@Master:~# kubectl get pods -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	aws-node-42rbw	2/2	Running	0	15m
kube-system	coredns-6976d5bf49-fvzkl	1/1	Running	0	15m
kube-system	coredns-6976d5bf49-gpvfz	1/1	Running	0	15m
kube-system	kube-proxy-zk6dt	1/1	Running	0	15m

- Verify Worker Node

```
root@Master:~# kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-19-10.ec2.internal	Ready	<none>	24m	v1.34.9-eks-8f14419

```
root@Master:~#
```

- Verify Cluster and namespaces

```
root@Master:~# kubectl cluster-info
Kubernetes control plane is running at https://EDB89D572F185064AC77DE71703A21CB.y14.us-east-1.eks.amazonaws.com
CoreDNS is running at https://EDB89D572F185064AC77DE71703A21CB.y14.us-east-1.eks.amazonaws.com/api/v1/namespaces/kube-
To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
root@Master:~# kubectl get ns
```

NAME	STATUS	AGE
default	Active	5h16m
kube-node-lease	Active	5h16m
kube-public	Active	5h16m
kube-system	Active	5h16m

2. setup eks cluster using console

- First create IAM Role for new cluster for EKS and add policy for it

Step 1
Select trusted entity

Step 2
Add permissions

Step 3
Name, review, and create

Select trusted entity Info

Trusted entity type

- ☒ **AWS service**
Allow AWS services like EC2, Lambda, or others to perform actions in this account.
- ☐ **AWS account**
Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.
- ☐ **Web identity**
Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.
- ☐ **SAML 2.0 federation**
Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.
- ☐ **Custom trust policy**
Create a custom trust policy to enable others to perform actions in this account.

Use case
Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case

EKS

- Select EKS Cluster

EKS

Choose a use case for the specified service.

Use case

- ☐ **EKS - Service**
Allows EKS to manage clusters on your behalf.
- ☒ **EKS - Cluster**
Allows the cluster Kubernetes control plane to manage AWS resources on your behalf.

- Given Role name and created cluster

Name, review, and create

Role details

Role name
Enter a meaningful name to identify this role.

EKS_Cluster_role

Maximum 64 characters. Use alphanumeric and '+=, @-_' characters.

Description
Add a short explanation for this role.

Allows the cluster Kubernetes control plane to manage AWS resources on your behalf.

- Add policy to it

Policies (1/1538) Info

A policy is an object in AWS that defines permissions.

Filter by Type

Q AmazonEksC X All types 3 matches

Policy name	Type	Used as	Description
☒ AmazonEKSClusterPolicy	AWS managed	Permissions policy (2)	This policy provides Kubernetes the per...
☐ AmazonEKSClusterPolicy	AWS managed	None	Policy attached to the EKS Cluster Role t...

- Now create role for Node groups

Step 1
Select trusted entity
 Step 2
 Add permissions
 Step 3
 Name, review, and create

Select trusted entity [Info](#)

Trusted entity type

☒ **AWS service**
 Allow AWS services like EC2, Lambda, or others to perform actions in this account.

☐ **AWS account**
 Allow entities in other AWS accounts belonging to you or a 3rd party to perform actions in this account.

☐ **Web identity**
 Allows users federated by the specified external web identity provider to assume this role to perform actions in this account.

☐ **SAML 2.0 federation**
 Allow users federated with SAML 2.0 from a corporate directory to perform actions in this account.

☐ **Custom trust policy**
 Create a custom trust policy to enable others to perform actions in this account.

Use case
 Allow an AWS service like EC2, Lambda, or others to perform actions in this account.

Service or use case
 EC2 ▼

- Given name as EKS-Node1 and create role

Name, review, and create

Role details

Role name
 Enter a meaningful name to identify this role.

EKS_Node1

Maximum 64 characters. Use alphanumeric and '+=, @-/_' characters.

Description
 Add a short explanation for this role.

Allows EC2 instances to call AWS services on your behalf.

Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: _+=, @-/_[]!#\$%^&*()~;"

- Now created New Cluster by selecting cluster IAM role and VPC

Cluster configuration

Name
 Use the auto-generated name or enter a unique name for this cluster. This property cannot be changed after the cluster is created.

Cluster_1

The cluster name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Max

Kubernetes version [Info](#)
 Select Kubernetes version for this cluster.

1.36 ▼

Cluster IAM role
 To access other AWS services and resources your Kubernetes control plane needs an IAM role. Choose a role with the right permissions for your Kubernetes cont

EKS_Cluster_role ▼ [Create new role](#)

[Edit role](#)

- Cluster created

Clusters (1)

Info

Last updated
July 29, 2026, 12:37 (UTC+05:30)

Delete

Create cluster

Q

Filter clusters

<

1

>

Cluster name ▲	Status ▼	Kubernetes version ▼	Support period ▼	Upgrade policy ▼	Created ▼	Provider
Cluster_1	Active	1.36	Standard support until August 2, 2027	Standard support	15 minutes ago	EKS

- Open cluster and open compute and create nodes add name and select node IAM role

Amazon Elastic Kubernetes Service > Clusters > [Cluster_1](#) > Add node group

Amazon Elastic Kubernetes Service

Dashboard

Clusters

Settings

Dashboard settings

Console settings

Amazon EKS Anywhere

Enterprise Subscriptions

Related services

Amazon ECR [i](#)

AWS Batch [i](#)

Documentation [i](#)

Step 1 **Configure node group**

Step 2 Set compute and scaling configuration

Step 3 Specify networking

Step 4 Review and create

Configure node group [Info](#)

A node group is a group of EC2 instances that supply compute capacity to your Amazon EKS cluster. You can add multiple node groups to your cluster.

Node group configuration

These properties cannot be changed after the node group is created.

Name

Assign a unique name for this node group.

The node group name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 63.

Node IAM role

To access other AWS services and resources your Nodes needs an IAM role. Choose a role with the right permissions for your Nodes.

[Refresh](#) [Create new role](#)

[Edit role](#)

- Check all details in scaling configuration

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type [Info](#)

Select the EKS-optimized Amazon Machine Image for nodes.

Capacity type

Select the capacity purchase option for this node group.

Instance types [Info](#)

Select instance types you prefer for this node group.

t3.micro [X](#)

vCPU: 2 vCPUs Memory: 1 GiB Network: Up to 5 Gigabit Max ENI: 2 Max IPs: 4

Disk size

Select the size of the attached EBS volume for each node.

GiB

- Select nodes desired and minimum and maximum sizes

Node group scaling configuration

Desired size

Set the desired number of nodes that the group should launch with initially.

nodes

Desired node size must be greater than or equal to 0

Minimum size

Set the minimum number of nodes that the group can scale in to.

nodes

Minimum node size must be greater than or equal to 0

Maximum size

Set the maximum number of nodes that the group can scale out to.

nodes

Maximum node size must be greater than or equal to 1 and cannot be lower than the minimum

- Select Subnets

Step 1
Configure node group

Step 2
Set compute and scaling configuration

Step 3
Specify networking

Step 4
Review and create

Specify networking

Node group network configuration

These properties cannot be changed after the node group is created.

Subnets [Info](#)
Specify the subnets in your VPC where your nodes will run. To create a new subnet, go to the corresponding page in the [VPC console](#).

Select subnets

subnet-038c9343c0d42d48d | eksctl-demo-eks-cluster-cluster/SubnetPrivateUSEAST1A
us-east-1a 192.168.96.0/19

subnet-005b14de348767415 | eksctl-demo-eks-cluster-cluster/SubnetPrivateUSEAST1B
us-east-1b 192.168.64.0/19

Clear selected subnets

☒ **Configure remote access to nodes** [Info](#)

- Now create Node group

etes Service > Clusters > Cluster_1 > Add node group

Step 1
Configure node group

Step 2
Set compute and scaling configuration

Step 3
Specify networking

Step 4
Review and create

Review and create

Step 1: Node group

Node group configuration

Name EKS_Node_Group	Node IAM role arn:aws:iam::3695037416
-------------------------------	---

- We can check nodes 2 worker nodes created

```
ubuntu@Master:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE     VERSION
ip-192-168-11-163.ec2.internal      Ready    <none>   2m13s   v1.34.9-eks-bca9cf6
ip-192-168-60-40.ec2.internal       Ready    <none>   2m14s   v1.34.9-eks-bca9cf6
ubuntu@Master:~$
```

3. Setup HPA

- Installed Metrics Server

```
ubuntu@Master:~$ eksctl create addon \
  --cluster demo-eks \
  --name metrics-server \
  --region us-east-1
2026-07-29 11:02:17 [i] Kubernetes version "1.34" in use by cluster "demo-eks"
2026-07-29 11:02:17 [i] creating addon: metrics-server
2026-07-29 11:03:20 [i] addon "metrics-server" active
ubuntu@Master:~$ kubectl get pods -n kube-system
```

```
ubuntu@Master:~$ kubectl get pods -n kube-system
NAME                                READY    STATUS    RESTARTS   AGE
aws-node-mpq2w                      2/2      Running   0           54m
aws-node-zsq7m                      2/2      Running   0           54m
coredns-857cbb5598-4gkrz            1/1      Running   0           57m
coredns-857cbb5598-hsmwb            1/1      Running   0           57m
kube-proxy-67v4x                    1/1      Running   0           54m
kube-proxy-vc78h                    1/1      Running   0           54m
metrics-server-744d76667f-4d2mk     1/1      Running   0           70s
metrics-server-744d76667f-nvg64     1/1      Running   0           70s
```

```
ubuntu@Master:~$ kubectl top nodes
NAME                                CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
ip-192-168-11-163.ec2.internal      39m          2%    399Mi           77%
ip-192-168-60-40.ec2.internal       28m          1%    390Mi           76%
ubuntu@Master:~$
```

- Created a Deployment

```
ubuntu@Master:~$ vi deployment.yaml
ubuntu@Master:~$
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hpa-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hpa-demo
  template:
    metadata:
      labels:
        app: hpa-demo
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: "100m"
            limits:
              cpu: "200m"
```


- Created a Service

```
ubuntu@Master:~$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-192-168-11-163.ec2.internal	Ready	<none>	72m	v1.34.9-eks-bca9cf6
ip-192-168-42-91.ec2.internal	Ready	<none>	72s	v1.34.9-eks-bca9cf6
ip-192-168-60-40.ec2.internal	Ready	<none>	72m	v1.34.9-eks-bca9cf6

```
ubuntu@Master:~$ kubectl top pods
```

NAME	CPU(cores)	MEMORY(bytes)
hpa-demo-66959c7696-dkxzz	0m	3Mi
hpa-demo-66959c7696-dws85	0m	4Mi

- Created an HPA

```
ubuntu@Master:~$ kubectl create deployment php-apache --image=registry.k8s.io/hpa-example
deployment.apps/php-apache created
ubuntu@Master:~$ 
ubuntu@Master:~$ kubectl set resources deployment php-apache \
--requests-cpu=200m
deployment.apps/php-apache resource requirements updated
ubuntu@Master:~$ kubectl expose deployment php-apache --port=80
service/php-apache exposed
ubuntu@Master:~$
```

- Autoscaling behaviour

```
ubuntu@Master:~$ kubectl autoscale deployment php-apache \
--cpu-percent=50 \
--min=1 \
--max=5
horizontalpodautoscaler.autoscaling/php-apache autoscaled
ubuntu@Master:~$
```

```
ubuntu@Master:~$ kubectl get hpa
NAME                REFERENCE                TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
hpa-demo            Deployment/hpa-demo      cpu: <unknown>/50%  2         5         2          25s
```

- Generated load

[illegible]

- HPA increased the replica count

```
ubuntu@Master:~$ kubectl get hpa -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	cpu: 0%/50%	1	5	1	90s
php-apache	Deployment/php-apache	cpu: 464%/50%	1	5	1	118s
php-apache	Deployment/php-apache	cpu: 297%/50%	1	5	4	2m13s

```
^Cubuntu@Master:~$
```

```
ubuntu@Master:~$ kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
load-generator	1/1	Running	0	34s
php-apache-6d768f465b-9rkzc	1/1	Running	0	2m4s
php-apache-6d768f465b-6474c	0/1	Pending	0	0s
php-apache-6d768f465b-b8fvf	0/1	Pending	0	0s
php-apache-6d768f465b-6474c	0/1	Pending	0	0s
php-apache-6d768f465b-fvs2x	0/1	Pending	0	0s
php-apache-6d768f465b-b8fvf	0/1	Pending	0	0s
php-apache-6d768f465b-fvs2x	0/1	Pending	0	0s
load-generator	0/1	Error	0	51s
load-generator	0/1	Error	0	52s

4. Setup cluster auto scale

- EKS Cluster Created

```
ubuntu@Master:~$ eksctl get cluster
```

NAME	REGION	EKSCTL CREATED
demo-eks	us-east-1	True

```
ubuntu@Master:~$
```

- Worker Nodes Running

```
ubuntu@Master:~$ kubectl get nodes -L node.kubernetes.io/instance-type
```

NAME	STATUS	ROLES	AGE	VERSION	INSTANCE-TYPE
ip-192-168-30-173.ec2.internal	Ready	<none>	17m	v1.34.9-eks-bca9cf6	t3.small
ip-192-168-31-117.ec2.internal	Ready	<none>	17m	v1.34.9-eks-bca9cf6	t3.small
ip-192-168-47-130.ec2.internal	Ready	<none>	17m	v1.34.9-eks-bca9cf6	t3.small

```
ubuntu@Master:~$
```

- Metrics Server Working

```
ubuntu@Master:~$ kubectl top nodes
```

NAME	CPU (cores)	CPU%	MEMORY (bytes)	MEMORY%
ip-192-168-30-173.ec2.internal	33m	1%	474Mi	33%
ip-192-168-31-117.ec2.internal	36m	1%	418Mi	29%
ip-192-168-47-130.ec2.internal	23m	1%	473Mi	33%

```
ubuntu@Master:~$
```

- IAM Service Account Created

```
ubuntu@Master:~$ eksctl get iamserviceaccount \
--cluster demo-eks \
--region us-east-1
```

NAMESPACE	NAME	ROLE ARN
kube-system	cluster-autoscaler	arn:aws:iam::369503741683:role/eksctl-demo-eks

```
ubuntu@Master:~$
```

- Service Account Annotation

```
ubuntu@Master:~$ kubectl get sa cluster-autoscaler -n kube-system -o yaml
```

```
apiVersion: v1
kind: ServiceAccount
metadata:
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::369503741683:role/eksctl-demo-eks
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","kind":"ServiceAccount","metadata":{"annotations":
name:"cluster-autoscaler","namespace":"kube-system"}}
```

- Cluster Autoscaler Running

```
ubuntu@Master:~$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS	AGE
aws-node-5gh7t	2/2	Running	0	2m32s
aws-node-7cl78	0/2	Pending	0	96s
aws-node-rzsjl	2/2	Running	0	26m
cluster-autoscaler-5c5b6447bf-tztx4	1/1	Running	0	13m
coredns-7c8bbf7487-88lqr	1/1	Running	0	2m40s

- Verify Autoscaler

```
ubuntu@Master:~$ kubectl get deployment -n kube-system cluster-autoscaler
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
cluster-autoscaler	1/1	1	1	25m

```
ubuntu@Master:~$
```

5. Setup cronjob and job

- Create job yaml file

```
ubuntu@Master:~$ nano job.yaml
```

```
ubuntu@Master:~$
```

```
GNU nano 8.7.1
```

```
apiVersion: batch/v1
kind: Job
metadata:
  name: demo-job
spec:
  template:
    spec:
      containers:
      - name: demo-job
        image: busybox
        command: ["sh", "-c", "echo Hello from Kubernetes Job && sleep 10"]
        restartPolicy: Never
      backoffLimit: 4
```

- Apply the Job and verify

```
ubuntu@Master:~$ kubectl apply -f job.yaml
```

```
job.batch/demo-job created
```

```
ubuntu@Master:~$ kubectl get jobs
```

NAME	STATUS	COMPLETIONS	DURATION	AGE
demo-job	Running	0/1	13s	13s

```
ubuntu@Master:~$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
demo-job-gcbng	0/1	Completed	0	20s

- Check logs

```
ubuntu@Master:~$ kubectl logs demo-job-gcbng
```

```
Hello from Kubernetes Job
```

- Create a Cronjob

```
ubuntu@Master:~$ nano cronjob.yaml
```

```
ubuntu@Master:~$
```

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: demo-cronjob
spec:
  schedule: "*/1 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: demo-cronjob
              image: busybox
              command:
                - /bin/sh
                - -c
                - date; echo "Hello from CronJob"
          restartPolicy: OnFailure

```

- Apply it and verify the Cronjob

```

ubuntu@Master:~$ kubectl apply -f cronjob.yaml
cronjob.batch/demo-cronjob created
ubuntu@Master:~$ kubectl get cronjobs
NAME          SCHEDULE    TIMEZONE    SUSPEND    ACTIVE    LAST SCHEDULE    AGE
demo-cronjob  */1 * * * * <none>      False      1          2s        11s
ubuntu@Master:~$ kubectl get jobs
NAME          STATUS    COMPLETIONS    DURATION    AGE
demo-cronjob-29755637  Complete  1/1            4s          75s
demo-cronjob-29755638  Complete  1/1            4s          15s
demo-job        Complete  1/1            14s         5m37s

```

- Check the pods

```

ubuntu@Master:~$ kubectl get pods
NAME                                READY    STATUS    RESTARTS    AGE
demo-cronjob-29755637-6lrfnf        0/1      Completed  0           87s
demo-cronjob-29755638-glrc5j        0/1      Completed  0           27s
demo-job-gcbrg                      0/1      Completed  0           5m49s
ubuntu@Master:~$ 

```

6. Create secret and inject inside pod

- Create the Secret

```

ubuntu@Master:~$ kubectl create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password=Password123
secret/db-secret created
ubuntu@Master:~$ 

```

- Verify the Secret

```

ubuntu@Master:~$ kubectl get secrets
NAME          TYPE    DATA    AGE
db-secret     Opaque  2        38s
ubuntu@Master:~$ 

```

- Create a Pod Using the Secret

```
ubuntu@Master:~$ nano secret-pod.yaml
ubuntu@Master:~$
```

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-pod
spec:
  containers:
  - name: app
    image: nginx
    env:
    - name: DB_USERNAME
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: username
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: password
```

- Deploy the Pod and verify

```
ubuntu@Master:~$ kubectl apply -f secret-pod.yaml
pod/secret-pod created
ubuntu@Master:~$ kubectl get pods
NAME                                READY    STATUS      RESTARTS   AGE
demo-cronjob-29755646-zrv7q         0/1      Completed   0           2m9s
demo-cronjob-29755647-s4szc         0/1      Completed   0           69s
demo-cronjob-29755648-dbmfh         0/1      Completed   0           9s
demo-job-gcbng                     0/1      Completed   0           15m
secret-pod                          1/1      Running     0           7s
ubuntu@Master:~$
```

- Verify the Secret Inside the Pod

```
ubuntu@Master:~$ kubectl exec secret-pod -- printenv DB_USERNAME
admin
ubuntu@Master:~$ kubectl exec secret-pod -- printenv DB_PASSWORD
Password123
ubuntu@Master:~$
```

7. Check different add-ons available on eks.

- List all installed add-ons

```
ubuntu@Master:~$ aws eks list-addons \
--cluster-name demo-eks \
--region us-east-1
{
  "addons": [
    "coredns",
    "kube-proxy",
    "metrics-server",
    "vpc-cni"
  ]
}
ubuntu@Master:~$
```

- Describe each add-on

```
ubuntu@Master:~$ aws eks describe-addon \
--cluster-name demo-eks \
--addon-name coredns \
--region us-east-1
{
  "addon": {
    "addonName": "coredns",
    "clusterName": "demo-eks",
    "status": "ACTIVE",
    "addonVersion": "v1.12.4-eksbuild.18",
    "health": {
      "issues": []
    },
    "addonArn": "arn:aws:eks:us-east-1:369503741683:addon/demo-eks/coredns",
    "createdAt": "2026-07-29T10:06:21.024000+00:00",
    "modifiedAt": "2026-07-29T14:41:33.374000+00:00",
    "tags": {},
    "namespaceConfig": {
      "namespace": "kube-system"
    }
  }
}
```

```
ubuntu@Master:~$ aws eks describe-addon \
--cluster-name demo-eks \
--addon-name kube-proxy \
--region us-east-1
{
  "addon": {
    "addonName": "kube-proxy",
    "clusterName": "demo-eks",
    "status": "ACTIVE",
    "addonVersion": "v1.34.6-eksbuild.17",
    "health": {
      "issues": []
    },
    "addonArn": "arn:aws:eks:us-east-1:369503741683:addon/demo-eks/kube-proxy",
    "createdAt": "2026-07-29T10:06:22.274000+00:00",
    "modifiedAt": "2026-07-29T10:06:28.359000+00:00",
    "tags": {},
    "namespaceConfig": {
      "namespace": "kube-system"
    }
  }
}
```

```
ubuntu@Master:~$ aws eks describe-addon \
--cluster-name demo-eks \
--addon-name vpc-cni \
--region us-east-1
{
  "addon": {
    "addonName": "vpc-cni",
    "clusterName": "demo-eks",
    "status": "ACTIVE",
    "addonVersion": "v1.22.3-eksbuild.1",
    "health": {
      "issues": []
    },
    "addonArn": "arn:aws:eks:us-east-1:369503741683:addon/demo-eks/vpc-cni",
    "createdAt": "2026-07-29T10:06:21.714000+00:00",
    "modifiedAt": "2026-07-29T10:06:28.263000+00:00",
    "tags": {},
    "namespaceConfig": {
      "namespace": "kube-system"
    }
  }
}
```

8. Upgrade eks cluster

- Let's check the current Kubernetes version

```
ubuntu@Master:~$ aws eks describe-cluster \
--name demo-eks \
--region us-east-1 \
--query "cluster.version"
"1.34"
ubuntu@Master:~$
```

- check the node group

```
ubuntu@Master:~$ eksctl get nodegroup \
--cluster demo-eks \
--region us-east-1
CLUSTER      NODEGROUP    STATUS  CREATED              MIN SIZE  MAX SIZE  DESIRED CAPACITY  INSTANCE TYPE
demo-eks     workers      ACTIVE  2026-07-29T14:39:15Z  1         5         1                 t3.small
rs-e4cfd728-5c72-039c-6c0a-0410808284c1 managed
ubuntu@Master:~$
ubuntu@Master:~$ eksctl upgrade cluster \
```

```
ubuntu@Master:~$
ubuntu@Master:~$ eksctl upgrade cluster \
--name demo-eks \
--region us-east-1
2026-07-29 15:39:09 [i] (plan) would upgrade cluster "demo-eks" control plane from current version "1.34" to "1.35"
2026-07-29 15:39:09 [i] re-building cluster stack "eksctl-demo-eks-cluster"
2026-07-29 15:39:09 [✓] all resources in cluster stack "eksctl-demo-eks-cluster" are up-to-date
2026-07-29 15:39:09 [i] checking security group configuration for all nodegroups
2026-07-29 15:39:09 [i] all nodegroups have up-to-date cloudformation templates
2026-07-29 15:39:09 [!] no changes were applied, run again with '--approve' to apply the changes
ubuntu@Master:~$
```

- Upgrade from 1.34 to 1.35

```
ubuntu@Master:~$ eksctl upgrade cluster \
--name demo-eks \
--region us-east-1 \
--approve
2026-07-29 15:40:20 [i] will upgrade cluster "demo-eks" control plane from current version "1.34" to "1.35"
2026-07-29 15:47:53 [✓] cluster "demo-eks" control plane has been upgraded to version "1.35"
2026-07-29 15:47:53 [i] you will need to follow the upgrade procedure for all of nodegroups and add-ons
2026-07-29 15:47:53 [i] re-building cluster stack "eksctl-demo-eks-cluster"
2026-07-29 15:47:53 [✓] all resources in cluster stack "eksctl-demo-eks-cluster" are up-to-date
2026-07-29 15:47:53 [i] checking security group configuration for all nodegroups
2026-07-29 15:47:53 [i] all nodegroups have up-to-date cloudformation templates
ubuntu@Master:~$ aws eks describe-cluster \
--name demo-eks \
--region us-east-1 \
--query "cluster.version"
"1.35"
```

- Verify nodes

```
ubuntu@Master:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-192-168-31-117.ec2.internal     Ready    <none>   70m   v1.34.9-eks-bca9cf6
ubuntu@Master:~$ aws eks describe-cluster \
--name demo-eks \
--region us-east-1 \
--query "cluster.version"
"1.35"
ubuntu@Master:~$
```